

四叶草平台五项目整合

项目整合总体架构设计说明书

项目	内容
版本	v1.1（基于确认版约束）
日期	2026-05-18
适用范围	Portal、合同审查、标书生成、RAG 问答、企业竞对分析
关键约束	PostgreSQL 18；第一阶段保留 iframe；暂用现有认证；数据短期共享；配置多环境化

0. 已确认约束

确认项	结论
PostgreSQL 版本	开发、测试、生产均统一使用 PostgreSQL 18。
认证体系	第一阶段复用现有 Portal 登录与 session 机制，平台稳定后再升级认证体系。
多租户	当前不做复杂多租户，但表结构预留 tenant_id / organization_id 等扩展口子。
标书生成服务边界	短期保留标书生成现有独立逻辑，中期合并为 bid_generator 模块。
任务队列	第一阶段不引入 Celery/RQ，后续按长任务压力再评估。
iframe 策略	第一阶段保留 iframe，第二阶段统一后端和数据库，第三阶段逐步去 iframe。
文件存储	暂时使用本地目录或 Docker volume，后续可平滑切换 MinIO。
权限策略	默认普通用户可使用五个模块，管理员可配置某用户不能使用某模块；所有数据短期共享。
数据隔离	暂时不做隔离，但业务表和查询接口预留 user_id、tenant_id、visibility、scope 等口子。
多环境配置	配置体系支持 dev/test/prod 多环境，Dify workflow key 按环境隔离。

1. 架构定位

平台采用“模块化单体优先、微服务后置”的架构策略。现阶段不拆微服务，不引入复杂服务治理，而是通过统一仓库、统一 FastAPI 后端、统一 PostgreSQL 18、统一配置体系和统一前端壳，降低维护成本。

架构层	设计结论
代码组织	monorepo, modules/ 按业务模块隔离, packages/ 存公共库。
后端形态	第一阶段保留现有服务，第二阶段收敛为一个 FastAPI 主应用。
前端形态	第一阶段保留 Portal iframe，第三阶段迁移为统一 React 路由。
数据库	PostgreSQL 18 单库多 schema；core 公共 schema + 业务 schema。
文件存储	本地 volume 优先，core.files 记录元数据，后续可切 MinIO。
认证	复用现有 Portal session，稳定后升级 access token + refresh token。

2. 目标架构图（文本版）

用户浏览器
-> apps/web 统一前端入口
 -> /apps/contract-review
 -> /apps/bid-generator

```
-> /apps/rag
-> /apps/competitor-analysis
-> /admin/users
-> apps/api 统一 FastAPI 后端
  -> core router: auth/users/permissions/files/jobs/settings
  -> contract_review router
  -> bid_generator router
  -> rag router
  -> competitor_analysis router
-> PostgreSQL 18: clover_platform
  -> core schema
  -> contract_review schema
  -> bid_generator schema
  -> rag schema
  -> competitor_analysis schema
-> 外部能力
  -> Dify workflows
  -> 本地文件 volume, 后续可替换 MinIO
```

3. 目录结构建议

```
clover-platform/
  apps/
    web/          # 统一前端入口
    api/          # 统一 FastAPI 后端入口
  modules/
    portal/
    contract_review/
    bid_generator/
    rag/
    competitor_analysis/
  packages/
    py_common/    # config/db/auth/dify/storage/jobs/logging
    ui/           # 前端公共组件
    api_client/   # 前端请求封装
    shared_types/ # 类型定义
  config/
    default.yaml
    apps.yaml
    workflows.yaml
    config.local.yaml # 不提交
  scripts/
    dev.py
    init_db.py
    migrate.py
  docker/
    docker-compose.yml
```

4. 架构演进边界

当前阶段	允许做	暂不做
阶段一	monorepo、统一启动、动态端口、配置集中、iframe 保留。	不大规模改业务逻辑；不强行统一 React 版本；不引入 Celery。
阶段二	PostgreSQL 18、统一后端、统一 Dify	不去 iframe；不做复杂租户隔离。

	client、统一文件元数据。	
阶段三	统一前端路由、公共 UI、统一 api-client、权限细化。	仍不强制微服务化。

5. 非功能性要求

- 可维护性：业务模块必须边界清晰，不允许跨模块直接读写对方表。
- 可回退性：每个阶段必须可以回退到上一阶段启动方式。
- 可观测性：API 请求、Dify 调用、后台任务、文件导出都要记录统一日志。
- 安全性：认证暂时复用现有机制，但所有 API 必须预留 user context。
- 可扩展性：表结构预留 tenant_id、organization_id、visibility、owner_id。